Offensive AI in Practice – Nov 2026

Hands-on exploitation of vulnerable applications using open source AI tools

2-hour workshop deck redesigned for paced facilitation, live coding, and agent runs

120 MIN **5 LABS** **HANDS-ON EXPERIENCE**

Audience: students running vulnerable apps locally in Docker, then comparing traditional scanning with AI-backed offensive workflows.



Workshop arc and pacing

Framing, then Shannon runs in the background while you work ZAP, PentestGPT, and ARTEX

Intro + trainer framing	5 min
Format + environment check	5 min
How AI changes offensive security	10 min
Lab 1 · Deploy Juice Shop	10 min
Lab 2 · Shannon agent (kick off)	**75 min** (runs in the background)
Lab 3 · ZAP baseline	**~20 min**
Lab 4 · PentestGPT	**~25 min**
Lab 5 · ARTEX	**~25 min**
Revisit Shannon + compare	20 min

Facilitation rhythm

Start Shannon first — it runs ~75 min in the background while you work Labs 3–5. ~135 min total lab time.

Outcomes and requirements

What students should leave with after the session

Learning outcomes

- Understand how AI compresses recon, analysis, and exploit iteration
- Run one traditional scanner and three AI-assisted offensive tools locally
- Compare the quality of findings, speed, and operator burden
- Recognize why operationalizing open source offensive AI is hard: cost, safety, and supply-chain trust

Student checklist

- Docker Desktop installed and validated
- Terminal access and Git installed
- Browser available for Juice Shop and dashboards
- Anthropic API key configured
- GitHub account ready for the workshop bundle

```
1 docker --version
2 docker run hello-world
3 export ANTHROPIC_API_KEY=your_key_here
```

Cost & model choice

Every AI tool bills your Anthropic key — pick cheaper models and a capped key

ZAP baseline	$0 — no API, free benchmark
PentestGPT	~$2 per run
ARTEX (Sonnet)	~$0.40 per run — Opus default is ~$3–4
Shannon lite	~$27, ~80 min — the big spend

Load ≥$40 credit on a dedicated, capped key. Set ARTEX_LLM_MODEL=claude-sonnet-4-6. ZAP is free.

How AI changes the attack loop

The key message: AI amplifies attacker capability, speed, and scale

Traditional model

- Reconnaissance
- Vulnerability discovery
- Exploit research or development
- Exploitation
- Privilege escalation or lateral movement

Often measured in hours to weeks.

AI-augmented model

- Endpoint discovery
- Parameter analysis
- Exploit generation
- Payload crafting, testing and iteration
- Vulnerability chaining

Machine-speed iteration compresses the cycle to minutes or hours.

Workshop tool stack

Open source tooling, commercial model backend, and containerized execution

Target

OWASP Juice Shop

- Modern SPA with login, APIs, and common web vulns

Traditional tool

OWASP ZAP baseline

- Fast benchmark for what a non-agent scan catches first

AI-assisted tools

1. PentestGPT
2. Shannon
3. ARTEX

- All run in containers and use Anthropic-backed reasoning

Delivery model: the repo provides bootstrap code, Docker Compose starts the app + tools, and each lab ends with a compare-and-debrief moment.

Architecture at a glance

One target, four tools, one Docker Compose stack — all local

Target

OWASP Juice Shop

- juice-shop:3000 — one shared container every tool points at

Four tools

ZAP · Shannon · PentestGPT · ARTEX

- One DAST scanner plus three AI agents, each in its own container

Shared & isolated

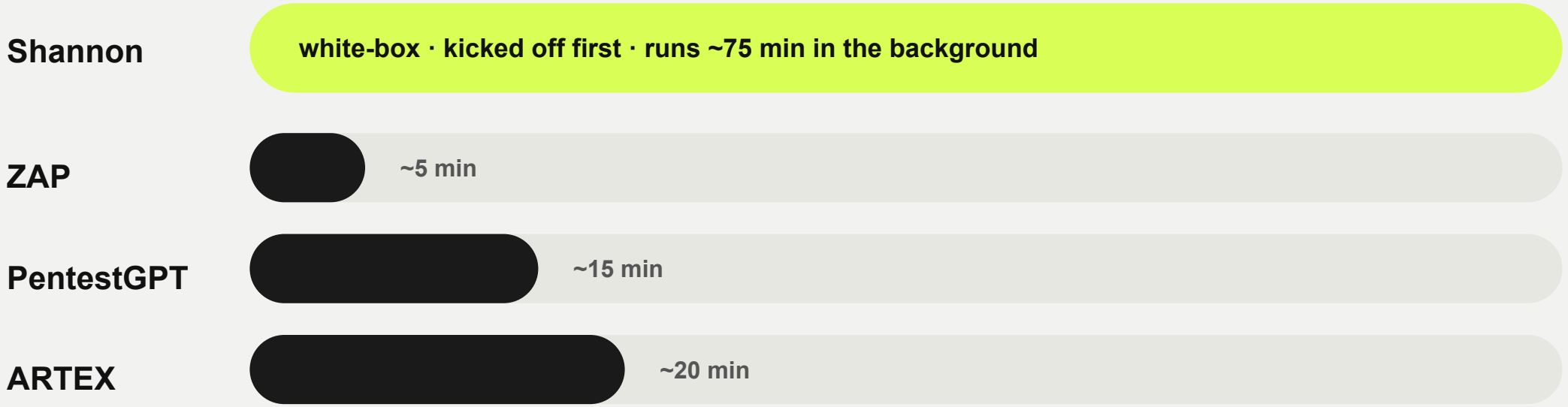
1. All share one Anthropic API key
2. Shannon and ARTEX each get their own Postgres

docker compose up -d brings up Juice Shop + the long-running services; ZAP, Shannon, PentestGPT, and ARTEX each run on demand.

Recommended running order

Start Shannon first — it is the long one — then run the rest while it works

ZAP, PentestGPT and ARTEX all finish inside Shannon's window — no interference



Separate outputs, dedicated Postgres for Shannon and ARTEX. They share only the target and your API budget.

Lab 1 · Deploy the vulnerable target

10 MIN

10-minute setup segment before the major labs

SETUP + VALIDATION

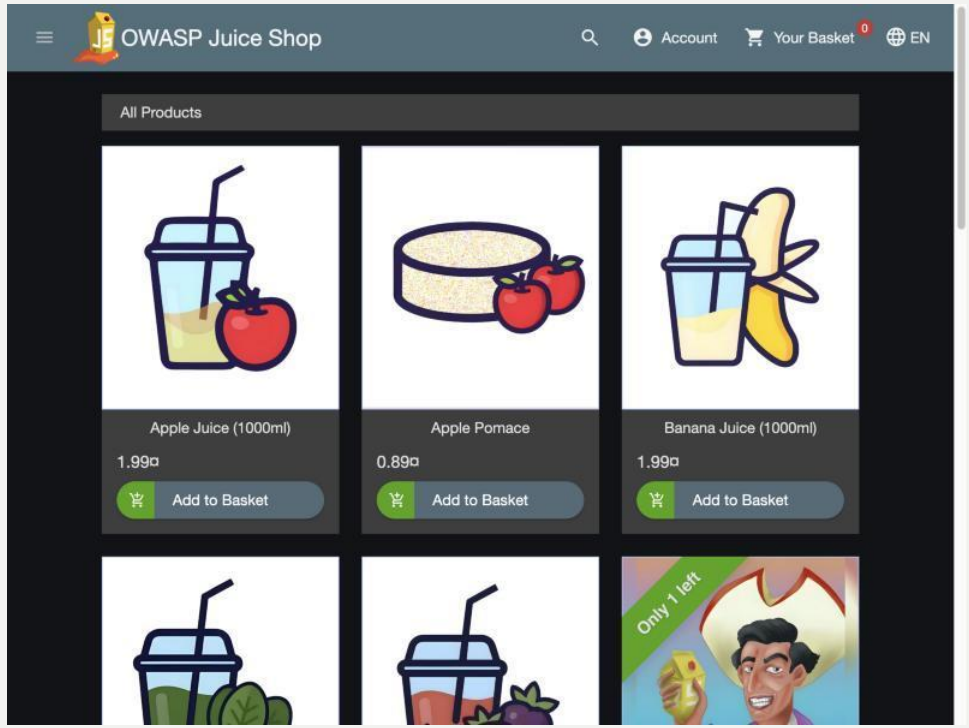
Target: OWASP Juice Shop

Modern single-page app

Intentionally vulnerable

- Good coverage for IDOR, auth flaws, injection, and access control issues

```
cd workshop-bundle
cp .env.example .env
# add ANTHROPIC_API_KEY
docker compose up -d
```



Confirm the app opens at localhost:3000 before moving on.

Lab 2 · Shannon agent workflow

75 MIN

We'll start by kicking Shannon off as it takes a while to run!

EXECUTION BLOCK 1

Why Shannon

Identifies insecure endpoints

Analyzes responses

- Generates exploit strategies and writes findings to a repo-backed workspace

```
1 docker compose ps
2 git clone
https://github.com/juice-shop/juice-shop.git \
3   workspace/juice-shop
```

```
1 docker compose run --rm shannon-worker node
apps/worker/dist/temporal/worker.js  \
2 http://juice-shop:3000 /app/repos/juice-shop
--task-queue shannon --config
/app/configs/juice-shop-lite.yaml
```


Keygraph

CONFIDENTIAL

Security Assessment Report

TARGET
http://juice-shop:3000

DATE
2026-10-27

STATUS
Complete

TESTER
Shannon
AI Pentester by Keygraph

This document contains sensitive security findings. Handle in accordance with your organization's data classification policy.

Lab 3 · ZAP baseline

30 MIN

Traditional DAST while Shannon runs so we can compare old with new

EXECUTION BLOCK 2

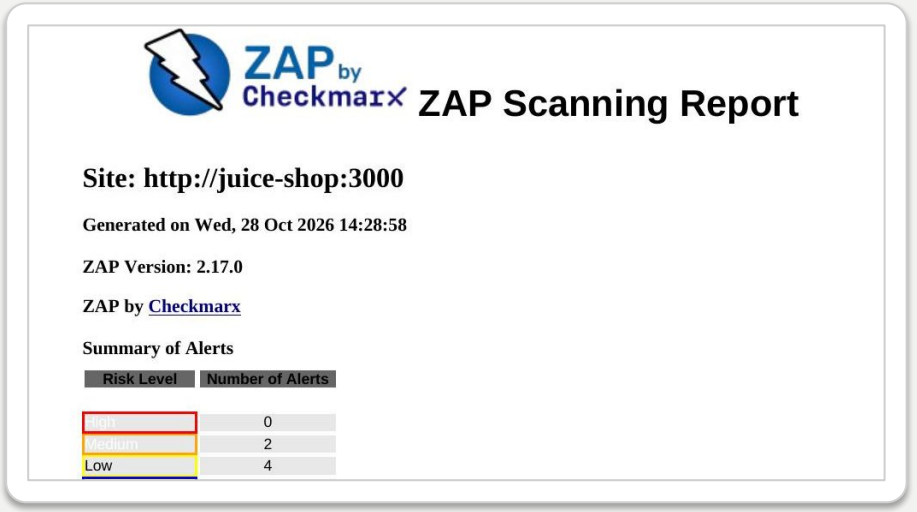
Run

```
docker compose run --rm zap-baseline
```

What to observe

- Baseline scanner output and HTML report
- Likely focus on straightforward endpoints and patterns
- Use a simple IDOR example to show where manual reasoning still matters

```
curl http://localhost:3000/rest/basket/2
```


ZAP by Checkmarx ZAP Scanning Report

Site: http://juice-shop:3000

Generated on Wed, 28 Oct 2026 14:28:58

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	4

Summary of Alerts — 0 High · 2 Medium · 4 Low · 3 Info

Debrief question: what did ZAP find quickly, and what still required analyst interpretation?

Lab 4 · PentestGPT

30 MIN

This lab emphasizes end-to-end pentest workflow assistance and generated commands

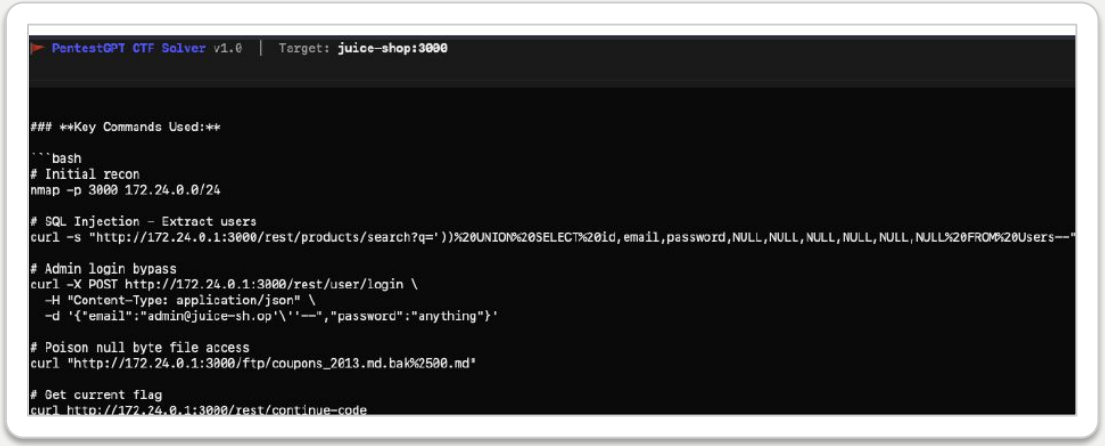
EXECUTION BLOCK 3

Run

```
1 docker compose exec -it pentestgpt bash
2 pentestgpt --target juice-shop:3000
--no-telemetry
```

What students compare

- Endpoint enumeration
- Suggested exploits
- Command generation
- How much analyst steering is still needed



Generated recon + SQLi / auth-bypass commands

Reminder: validate findings inside Juice Shop rather than treating model output as automatically correct.

Lab 5 · ARTEX

~25 MIN

Supervised autonomous agent with a dashboard — you watch and steer, not just read logs

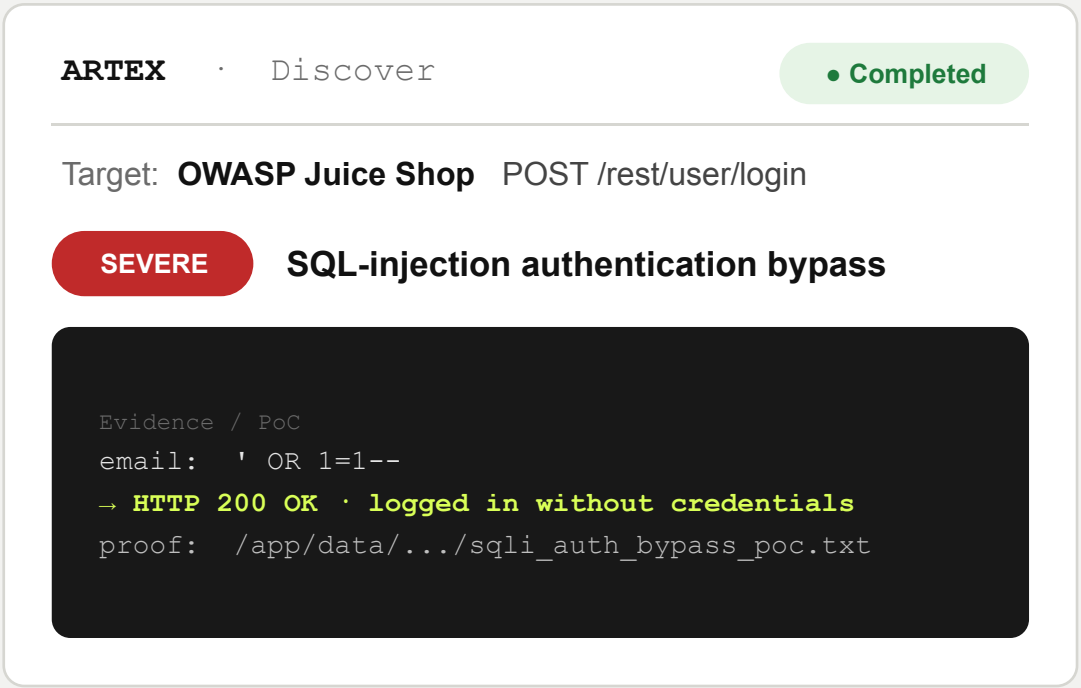
EXECUTION BLOCK 4

Run

```
1 docker compose --profile artex up -d artex
2 open http://localhost:8787   # set a
password, then New task
```

What to do

- Description = the target URL · Target = the goal
- Watch the Session, Tool execution, and Discover tabs
- Approve tool calls at the human-in-the-loop gate



Revisit Shannon output

You kicked it off first — now review the report the agent wrote while you worked

What Shannon produced

- Per-vulnerability findings: injection, XSS, auth, authz, SSRF
- Exploitation queues + reconciliation manifests
- A comprehensive report (Markdown + PDF)
- Git checkpoints in the target repo

Debrief

workspace/juice-shop/.shannon/deliverables/

- Source-driven depth vs ZAP's baseline and PentestGPT's walkthrough
- It ran unattended while you worked Labs 3–5
- Which workflow would you trust first — and which gave the most analyst leverage?

Cross-lab comparison

Use this slide to debrief after the final coding block

Dimension	ZAP baseline	PentestGPT	Shannon	ARTEX
Setup burden	Low	Medium	Medium	High
Autonomy	Low	High	High	High
Output style	Scanner report	Interactive guidance	Repo + workflow traces	Dashboard + findings
Best teaching use	Baseline and control	Operator in the loop	Reasoning over app behavior	Supervised agent + GUI

Thought experiment: Which workflow would you trust first in a real assessment, and which one created the most useful analyst leverage?

ARTEX in the wild — why we study it

Reuters World Business Markets Sustainability Legal Commentary Technology Investigations More

Artificial Intelligence > Anthropic IPO | Rogue agents | PHP flaws

Chinese developer makes ARTEX AI agent closed-source after Korean bank hack

By **Brenda Goh**

October 8, 2026 9:56 PM EDT · Updated October 8, 2026

Top stories Customize

BleepingComputer
Former pentester used ARTEX AI to steal 8.2 million data records
1 day ago



TechCrunch
Japan internet giant GMO says hackers posted stolen customer data
1 day ago



WSJ
Chinese AI Tool Linked to Hacks in Japan
3 hours ago



Zscaler
Inside a Global AI-Assisted Hacking Campaign | ThreatLabz
6 days ago

A financially-motivated actor paired ARTEX with LLMs (DeepSeek, GLM-5.3, Grok) to breach South Korean banks and exfiltrate data — late Sept to early Oct 2026.

The tools in this lab are the ones being turned on defenders. Knowing how they work — and their supply-chain risk — is the job.

Why open source offensive AI is hard to scale

Infrastructure complexity	GPU compute, scaling, API management
Safety	Open source models can be less consistent and require guardrails
Enterprise requirements	Audit logging, authentication, RBAC, compliance
Operational overhead	Containers, models, updates, and security risk management

Final wrap-up

Leave students with the attacker model, not just the tools

Key takeaways

AI is transforming offensive security by collapsing recon and exploit iteration time

Traditional scanning still matters as a baseline and validation layer

Agentic tools — PentestGPT, Shannon, ARTEX — accelerate workflows, but need oversight and supply-chain trust

Defenders need hands-on familiarity with offensive AI and its trust trade-offs

Q&A · 5 MIN

“What changed most for you today: speed, confidence, or the amount of manual work?”